In [4]:
```
import pandas as pd
import numpy as np

# Load the main dataset
data = pd.read_csv(r"C:\Users\Manaswini\Downloads\adst\Training.csv")

# Display first 5 rows
print(data.head())

# Display shape
print("Dataset Shape:", data.shape)

# Display column names
print("\nColumns:")
print(data.columns.tolist())

# Check missing values
print("\nMissing Values:")
print(data.isnull().sum().sum())

# Check duplicate rows
print("\nDuplicate Rows:", data.duplicated().sum())

# Number of diseases
print("\nNumber of Diseases:", data["prognosis"].nunique())

# Disease names
print("\nDiseases:")
print(data["prognosis"].unique())
```

```
   itching  skin_rash  nodal_skin_eruptions  continuous_sneezing  shivering  \
0        1          1                     1                    0          0
1        0          1                     1                    0          0
2        1          0                     1                    0          0
3        1          1                     0                    0          0
4        1          1                     1                    0          0

   chills  joint_pain  stomach_pain  acidity  ulcers_on_tongue  ...  \
0       0           0             0        0                 0  ...
1       0           0             0        0                 0  ...
2       0           0             0        0                 0  ...
3       0           0             0        0                 0  ...
4       0           0             0        0                 0  ...

   blackheads  scurring  skin_peeling  silver_like_dusting  \
0           0         0             0                    0
1           0         0             0                    0
2           0         0             0                    0
3           0         0             0                    0
4           0         0             0                    0

   small_dents_in_nails  inflammatory_nails  blister  red_sore_around_nose  \
0                     0                   0        0                     0
1                     0                   0        0                     0
2                     0                   0        0                     0
3                     0                   0        0                     0
4                     0                   0        0                     0

   yellow_crust_ooze         prognosis
0                  0  Fungal infection
1                  0  Fungal infection
2                  0  Fungal infection
3                  0  Fungal infection
4                  0  Fungal infection

[5 rows x 133 columns]
Dataset Shape: (4920, 133)

Columns:
['itching', 'skin_rash', 'nodal_skin_eruptions', 'continuous_sneezing', 'shivering', 'chills', 'joint_pain', 'stomach_pain', 'acidity', 'ulcers_on_tongue', 'muscle_wasting', 'vomiting', 'burni
ng_micturition', 'spotting_ urination', 'fatigue', 'weight_gain', 'anxiety', 'cold_hands_and_feets', 'mood_swings', 'weight_loss', 'restlessness', 'lethargy', 'patches_in_throat', 'irregular_s
ugar_level', 'cough', 'high_fever', 'sunken_eyes', 'breathlessness', 'sweating', 'dehydration', 'indigestion', 'headache', 'yellowish_skin', 'dark_urine', 'nausea', 'loss_of_appetite', 'pain_b
ehind_the_eyes', 'back_pain', 'constipation', 'abdominal_pain', 'diarrhoea', 'mild_fever', 'yellow_urine', 'yellowing_of_eyes', 'acute_liver_failure', 'fluid_overload', 'swelling_of_stomach',
 'swelled_lymph_nodes', 'malaise', 'blurred_and_distorted_vision', 'phlegm', 'throat_irritation', 'redness_of_eyes', 'sinus_pressure', 'runny_nose', 'congestion', 'chest_pain', 'weakness_in_lim
bs', 'fast_heart_rate', 'pain_during_bowel_movements', 'pain_in_anal_region', 'bloody_stool', 'irritation_in_anus', 'neck_pain', 'dizziness', 'cramps', 'bruising', 'obesity', 'swollen_legs',
 'swollen_blood_vessels', 'puffy_face_and_eyes', 'enlarged_thyroid', 'brittle_nails', 'swollen_extremeties', 'excessive_hunger', 'extra_marital_contacts', 'drying_and_tingling_lips', 'slurred_s
peech', 'knee_pain', 'hip_joint_pain', 'muscle_weakness', 'stiff_neck', 'swelling_joints', 'movement_stiffness', 'spinning_movements', 'loss_of_balance', 'unsteadiness', 'weakness_of_one_body_
side', 'loss_of_smell', 'bladder_discomfort', 'foul_smell_of urine', 'continuous_feel_of_urine', 'passage_of_gases', 'internal_itching', 'toxic_look_(typhos)', 'depression', 'irritability', 'm
uscle_pain', 'altered_sensorium', 'red_spots_over_body', 'belly_pain', 'abnormal_menstruation', 'dischromic _patches', 'watering_from_eyes', 'increased_appetite', 'polyuria', 'family_history',
 'mucoid_sputum', 'rusty_sputum', 'lack_of_concentration', 'visual_disturbances', 'receiving_blood_transfusion', 'receiving_unsterile_injections', 'coma', 'stomach_bleeding', 'distention_of_abd
omen', 'history_of_alcohol_consumption', 'fluid_overload.1', 'blood_in_sputum', 'prominent_veins_on_calf', 'palpitations', 'painful_walking', 'pus_filled_pimples', 'blackheads', 'scurring', 's
kin_peeling', 'silver_like_dusting', 'small_dents_in_nails', 'inflammatory_nails', 'blister', 'red_sore_around_nose', 'yellow_crust_ooze', 'prognosis']

Missing Values:
0

Duplicate Rows: 4616

Number of Diseases: 41

Diseases:
<ArrowStringArray>
[                       'Fungal infection',
                                'Allergy',
                                   'GERD',
                        'Chronic cholestasis',
                              'Drug Reaction',
                        'Peptic ulcer diseae',
                                   'AIDS',
                                'Diabetes ',
                           'Gastroenteritis',
                          'Bronchial Asthma',
                             'Hypertension ',
                                 'Migraine',
                       'Cervical spondylosis',
                'Paralysis (brain hemorrhage)',
                                 'Jaundice',
                                  'Malaria',
                               'Chicken pox',
                                   'Dengue',
                                  'Typhoid',
                               'hepatitis A',
                               'Hepatitis B',
                               'Hepatitis C',
                               'Hepatitis D',
                               'Hepatitis E',
                        'Alcoholic hepatitis',
                              'Tuberculosis',
                               'Common Cold',
                                 'Pneumonia',
                'Dimorphic hemmorhoids(piles)',
                              'Heart attack',
                            'Varicose veins',
                            'Hypothyroidism',
                           'Hyperthyroidism',
                              'Hypoglycemia',
                           'Osteoarthristis',
                                'Arthritis',
                 '(vertigo) Paroymsal  Positional Vertigo',
                                    'Acne',
                           'Urinary tract infection',
                                'Psoriasis',
                                 'Impetigo']
Length: 41, dtype: str
```

In [5]:
```
# ==============================
# STEP 2: DATA PREPROCESSING
# ==============================

# Check missing values
print("Total missing values:", data.isnull().sum().sum())

# Check duplicate rows
print("Total duplicate rows:", data.duplicated().sum())

# Check data types
print("\nData Types:")
print(data.dtypes.value_counts())

# Separate features and target
X = data.drop("prognosis", axis=1)
y = data["prognosis"]

print("\nFeature shape:", X.shape)
print("Target shape:", y.shape)
```

```
# Number of unique diseases
print("\nNumber of diseases:", y.nunique())

# Disease distribution
print("\nDisease distribution:")
print(y.value_counts())
```

```
Total missing values: 0
Total duplicate rows: 4616

Data Types:
int64    132
str        1
Name: count, dtype: int64

Feature shape: (4920, 132)
Target shape: (4920,)

Number of diseases: 41

Disease distribution:
prognosis
Fungal infection                           120
Allergy                                    120
GERD                                       120
Chronic cholestasis                        120
Drug Reaction                              120
Peptic ulcer diseae                        120
AIDS                                       120
Diabetes                                   120
Gastroenteritis                            120
Bronchial Asthma                           120
Hypertension                               120
Migraine                                   120
Cervical spondylosis                       120
Paralysis (brain hemorrhage)               120
Jaundice                                   120
Malaria                                    120
Chicken pox                                120
Dengue                                     120
Typhoid                                    120
hepatitis A                                120
Hepatitis B                                120
Hepatitis C                                120
Hepatitis D                                120
Hepatitis E                                120
Alcoholic hepatitis                        120
Tuberculosis                               120
Common Cold                                120
Pneumonia                                  120
Dimorphic hemmorhoids(piles)               120
Heart attack                               120
Varicose veins                             120
Hypothyroidism                             120
Hyperthyroidism                            120
Hypoglycemia                               120
Osteoarthristis                            120
Arthritis                                  120
(vertigo) Paroymsal  Positional Vertigo    120
Acne                                       120
Urinary tract infection                    120
Psoriasis                                  120
Impetigo                                   120
Name: count, dtype: int64
```

In [6]:
```
import matplotlib.pyplot as plt
import seaborn as sns

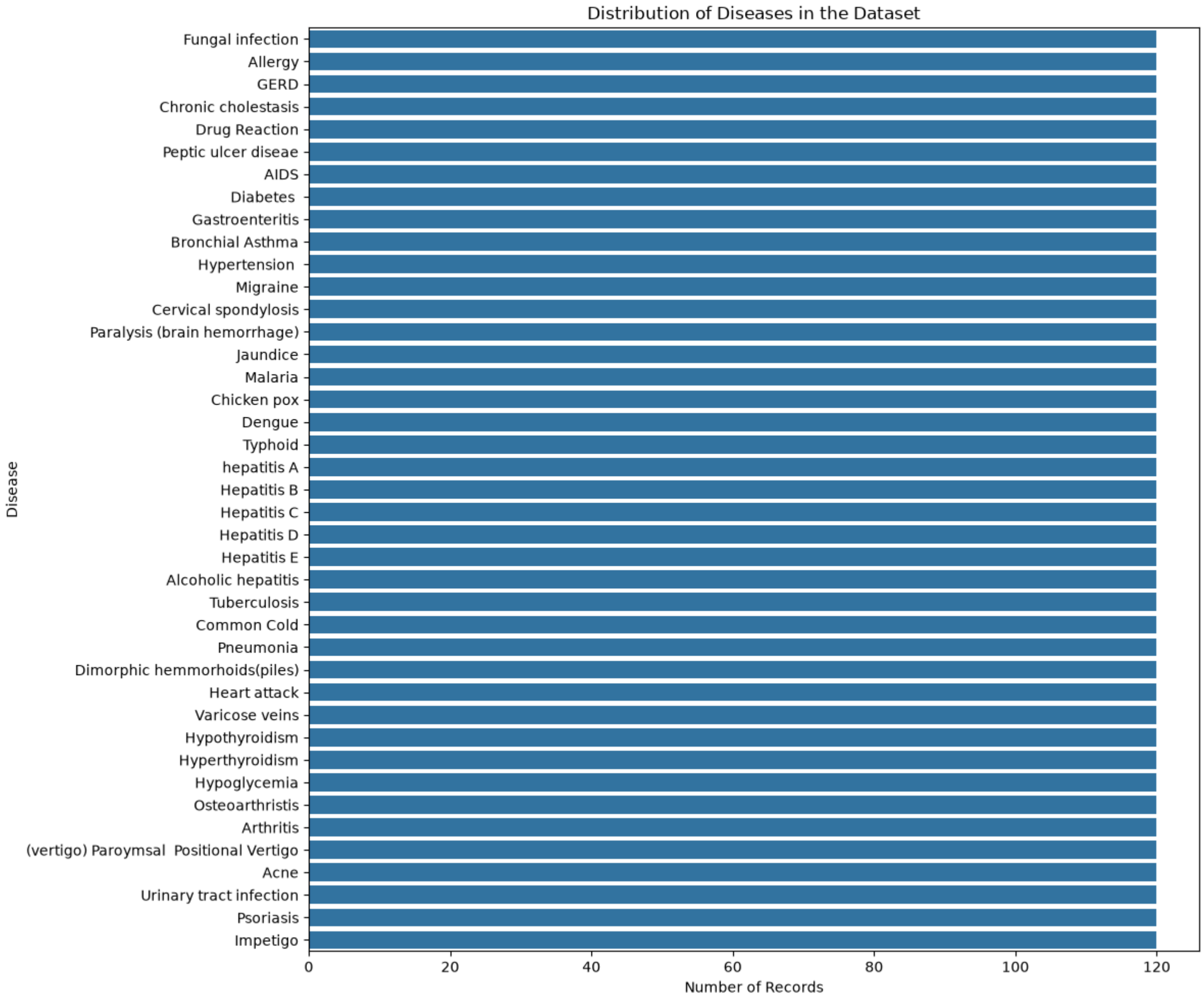
# Count diseases
disease_counts = y.value_counts()

plt.figure(figsize=(12, 10))

sns.barplot(
    x=disease_counts.values,
    y=disease_counts.index
)

plt.title("Distribution of Diseases in the Dataset")
plt.xlabel("Number of Records")
plt.ylabel("Disease")

plt.tight_layout()
plt.show()
```



In [7]:
```python
from sklearn.preprocessing import LabelEncoder

# Create label encoder
label_encoder = LabelEncoder()

# Encode disease names
y_encoded = label_encoder.fit_transform(y)

print("Original diseases:")
print(y.unique()[:10])

print("\nEncoded values:")
print(y_encoded[:10])

print("\nNumber of classes:", len(label_encoder.classes_))
```

```
Original diseases:
<ArrowStringArray>
[   'Fungal infection',             'Allergy',                'GERD',
 'Chronic cholestasis',       'Drug Reaction', 'Peptic ulcer diseae',
                'AIDS',           'Diabetes ',     'Gastroenteritis',
    'Bronchial Asthma']
Length: 10, dtype: str

Encoded values:
[15 15 15 15 15 15 15 15 15 15]

Number of classes: 41
```

In [8]:
```python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y_encoded,
    test_size=0.20,
    random_state=42,
    stratify=y_encoded
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (3936, 132)
Testing data: (984, 132)
```

In [10]:
```python
# ========================================
# STEP 6: MACHINE LEARNING MODEL TRAINING
# ========================================

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Create models
models = {
    "Logistic Regression": LogisticRegression(max_iter=2000),

    "Random Forest": RandomForestClassifier(
        n_estimators=200,
        random_state=42
    ),

    "Support Vector Machine": SVC(
        random_state=42
    ),
```

```
    "K-Nearest Neighbors": KNeighborsClassifier(
        n_neighbors=5
    )
}

# Dictionary to store results
model_results = {}

# Train and evaluate each model
for name, model in models.items():

    print("\nTraining:", name)

    # Train model
    model.fit(X_train, y_train)

    # Predict test data
    predictions = model.predict(X_test)

    # Calculate accuracy
    accuracy = accuracy_score(y_test, predictions)

    model_results[name] = accuracy

    print("Accuracy:", round(accuracy * 100, 2), "%")
```

```
Training: Logistic Regression
Accuracy: 100.0 %

Training: Random Forest
Accuracy: 100.0 %

Training: Support Vector Machine
Accuracy: 100.0 %

Training: K-Nearest Neighbors
Accuracy: 100.0 %
```

In [11]:

```
# ========================================
# STEP 7: MODEL COMPARISON
# ========================================

import matplotlib.pyplot as plt

model_names = list(model_results.keys())
accuracies = list(model_results.values())

plt.figure(figsize=(10, 6))

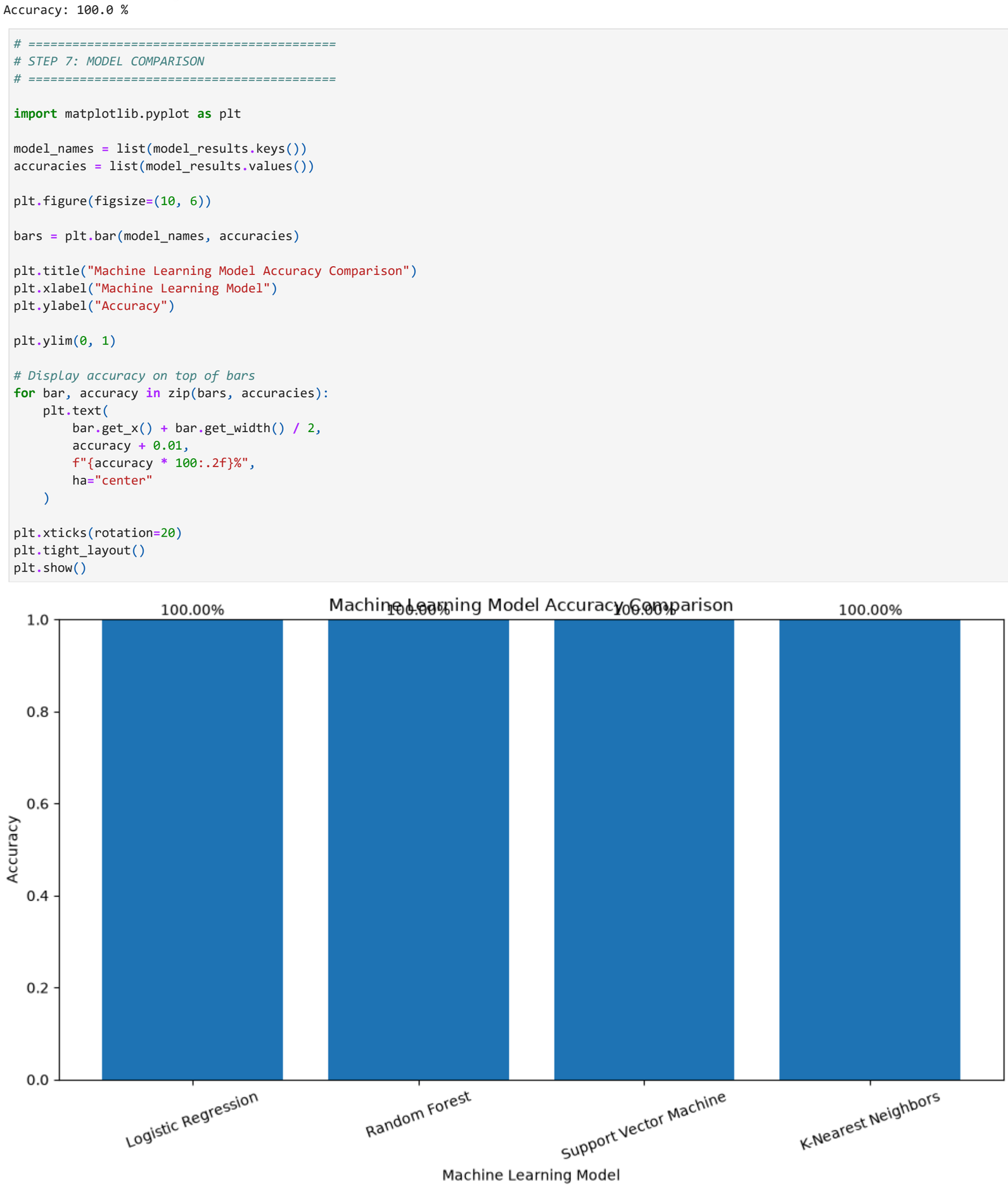
bars = plt.bar(model_names, accuracies)

plt.title("Machine Learning Model Accuracy Comparison")
plt.xlabel("Machine Learning Model")
plt.ylabel("Accuracy")

plt.ylim(0, 1)

# Display accuracy on top of bars
for bar, accuracy in zip(bars, accuracies):
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        accuracy + 0.01,
        f"{accuracy * 100:.2f}%",
        ha="center"
    )

plt.xticks(rotation=20)
plt.tight_layout()
plt.show()
```



In [12]:

```
# ========================================
# STEP 8: SELECT BEST MODEL
# ========================================

best_model_name = max(model_results, key=model_results.get)
best_accuracy = model_results[best_model_name]

print("Best Model:", best_model_name)
print("Best Accuracy:", round(best_accuracy * 100, 2), "%")
```

```
Best Model: Logistic Regression
Best Accuracy: 100.0 %
```

In [13]:

```
# ========================================
# STEP 9: DETAILED MODEL EVALUATION
# ========================================

from sklearn.metrics import (
    accuracy_score,
    precision_score,
```

```
    recall_score,
    f1_score,
    classification_report
)

# Get the best model
best_model = models[best_model_name]

# Predict test data
y_pred = best_model.predict(X_test)

# Calculate evaluation metrics
accuracy = accuracy_score(y_test, y_pred)

precision = precision_score(
    y_test,
    y_pred,
    average="weighted",
    zero_division=0
)

recall = recall_score(
    y_test,
    y_pred,
    average="weighted",
    zero_division=0
)

f1 = f1_score(
    y_test,
    y_pred,
    average="weighted",
    zero_division=0
)

print("Model Evaluation Results")
print("------------------------")
print("Model:", best_model_name)
print("Accuracy :", round(accuracy * 100, 2), "%")
print("Precision:", round(precision * 100, 2), "%")
print("Recall   :", round(recall * 100, 2), "%")
print("F1 Score :", round(f1 * 100, 2), "%")
```

```
Model Evaluation Results
------------------------
Model: Logistic Regression
Accuracy : 100.0 %
Precision: 100.0 %
Recall   : 100.0 %
F1 Score : 100.0 %
```

In [14]:
```
# Detailed classification report

print(classification_report(
    y_test,
    y_pred,
    target_names=label_encoder.classes_,
    zero_division=0
))
```

```
                                  precision    recall  f1-score   support

(vertigo) Paroymsal  Positional Vertigo       1.00      1.00      1.00        24
                                AIDS       1.00      1.00      1.00        24
                                Acne       1.00      1.00      1.00        24
                Alcoholic hepatitis       1.00      1.00      1.00        24
                             Allergy       1.00      1.00      1.00        24
                           Arthritis       1.00      1.00      1.00        24
                      Bronchial Asthma       1.00      1.00      1.00        24
                  Cervical spondylosis       1.00      1.00      1.00        24
                           Chicken pox       1.00      1.00      1.00        24
                  Chronic cholestasis       1.00      1.00      1.00        24
                           Common Cold       1.00      1.00      1.00        24
                                Dengue       1.00      1.00      1.00        24
                              Diabetes       1.00      1.00      1.00        24
Dimorphic hemmorhoids(piles)       1.00      1.00      1.00        24
                        Drug Reaction       1.00      1.00      1.00        24
                      Fungal infection       1.00      1.00      1.00        24
                                  GERD       1.00      1.00      1.00        24
                      Gastroenteritis       1.00      1.00      1.00        24
                         Heart attack       1.00      1.00      1.00        24
                           Hepatitis B       1.00      1.00      1.00        24
                           Hepatitis C       1.00      1.00      1.00        24
                           Hepatitis D       1.00      1.00      1.00        24
                           Hepatitis E       1.00      1.00      1.00        24
                          Hypertension       1.00      1.00      1.00        24
                      Hyperthyroidism       1.00      1.00      1.00        24
                         Hypoglycemia       1.00      1.00      1.00        24
                       Hypothyroidism       1.00      1.00      1.00        24
                             Impetigo       1.00      1.00      1.00        24
                             Jaundice       1.00      1.00      1.00        24
                              Malaria       1.00      1.00      1.00        24
                             Migraine       1.00      1.00      1.00        24
                      Osteoarthristis       1.00      1.00      1.00        24
Paralysis (brain hemorrhage)       1.00      1.00      1.00        24
                 Peptic ulcer diseae       1.00      1.00      1.00        24
                            Pneumonia       1.00      1.00      1.00        24
                            Psoriasis       1.00      1.00      1.00        24
                         Tuberculosis       1.00      1.00      1.00        24
                              Typhoid       1.00      1.00      1.00        24
                Urinary tract infection       1.00      1.00      1.00        24
                        Varicose veins       1.00      1.00      1.00        24
                           hepatitis A       1.00      1.00      1.00        24

                              accuracy                           1.00       984
                             macro avg       1.00      1.00      1.00       984
                          weighted avg       1.00      1.00      1.00       984
```

In [16]:
```
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

# Create confusion matrix
cm = confusion_matrix(y_test, y_pred)

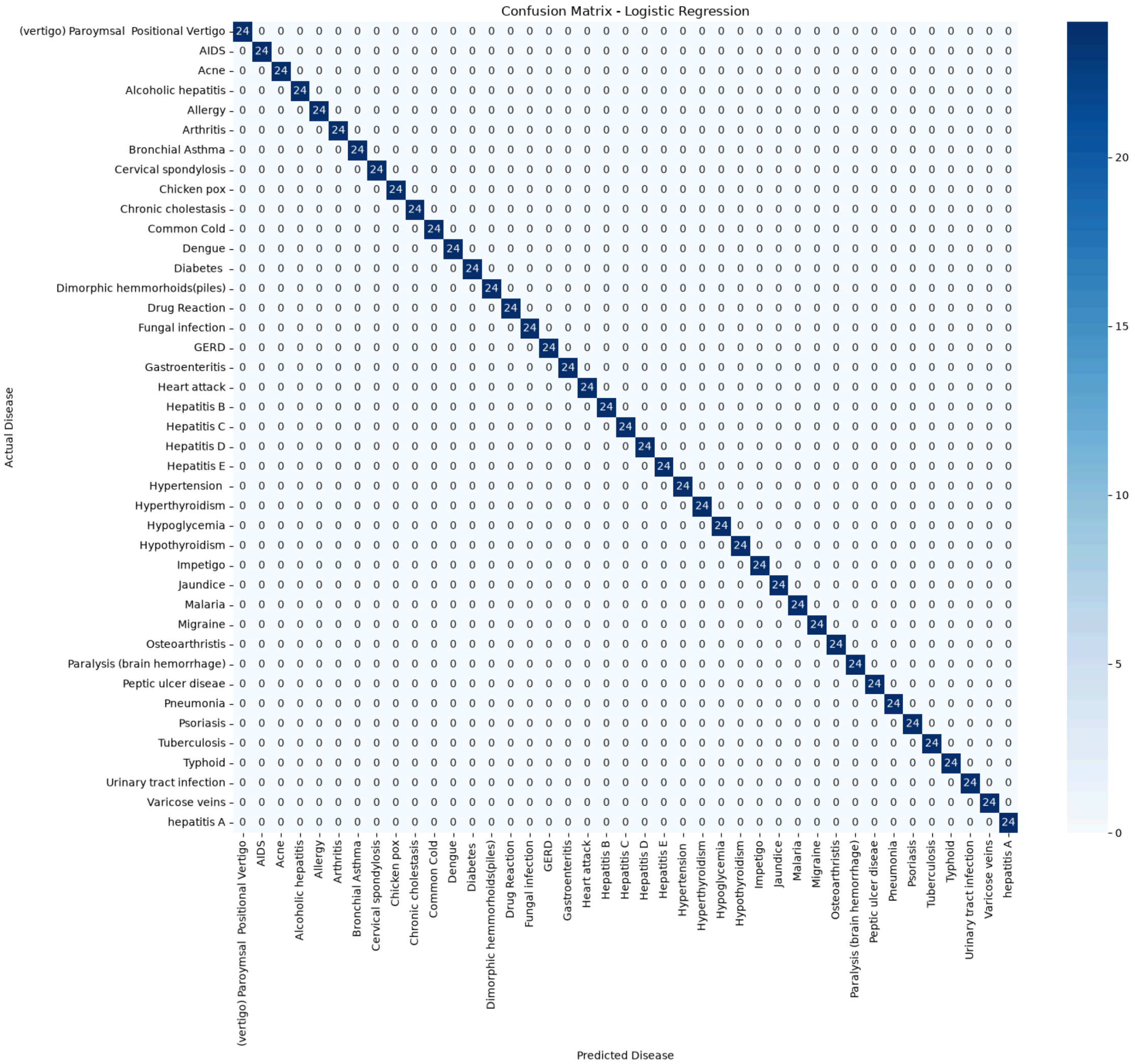
plt.figure(figsize=(16, 14))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=label_encoder.classes_,
    yticklabels=label_encoder.classes_
)
```

```
plt.title("Confusion Matrix - Logistic Regression")
plt.xlabel("Predicted Disease")
plt.ylabel("Actual Disease")

plt.xticks(rotation=90)
plt.yticks(rotation=0)

plt.tight_layout()
plt.show()
```



In [17]:
```
# ========================================
# STEP 12A: GET SYMPTOM LIST
# ========================================

# Get all symptom names from the training dataset
symptom_columns = X.columns.tolist()

print("Total symptoms:", len(symptom_columns))

print("\nFirst 20 symptoms:")
print(symptom_columns[:20])
```

```
Total symptoms: 132

First 20 symptoms:
['itching', 'skin_rash', 'nodal_skin_eruptions', 'continuous_sneezing', 'shivering', 'chills', 'joint_pain', 'stomach_pain', 'acidity', 'ulcers_on_tongue', 'muscle_wasting', 'vomiting', 'burni
ng_micturition', 'spotting_ urination', 'fatigue', 'weight_gain', 'anxiety', 'cold_hands_and_feets', 'mood_swings', 'weight_loss']
```

In [18]:
```
# ========================================
# STEP 12B: CREATE SYMPTOM MAPPING
# ========================================

symptom_mapping = {}

for symptom in symptom_columns:
    readable_name = symptom.replace("_", " ").lower()
    symptom_mapping[readable_name] = symptom

# Display some examples
for key, value in list(symptom_mapping.items())[:20]:
    print(key, " --> ", value)
```

```
itching  -->  itching
skin rash  -->  skin_rash
nodal skin eruptions  -->  nodal_skin_eruptions
continuous sneezing  -->  continuous_sneezing
shivering  -->  shivering
chills  -->  chills
joint pain  -->  joint_pain
stomach pain  -->  stomach_pain
acidity  -->  acidity
ulcers on tongue  -->  ulcers_on_tongue
muscle wasting  -->  muscle_wasting
vomiting  -->  vomiting
burning micturition  -->  burning_micturition
spotting  urination  -->  spotting_ urination
fatigue  -->  fatigue
weight gain  -->  weight_gain
anxiety  -->  anxiety
cold hands and feets  -->  cold_hands_and_feets
mood swings  -->  mood_swings
weight loss  -->  weight_loss
```

In [24]:
```
# ========================================
# STEP 12C: NLP SYMPTOM EXTRACTION
# ========================================

import re

def extract_symptoms(user_text):
    text = user_text.lower()
    text = re.sub(r"[^a-zA-Z\s]", " ", text)

    detected_symptoms = []

    for readable_name, original_name in symptom_mapping.items():

        # Direct matching
        if readable_name in text:
            detected_symptoms.append(original_name)

        # Fever variations
        elif "fever" in readable_name and "fever" in text:
            detected_symptoms.append(original_name)

    return list(set(detected_symptoms))
```

In [25]:
```
# ========================================
# STEP 12D: TEST NLP
# ========================================

user_input = "I have fever, headache, chills and joint pain"

detected = extract_symptoms(user_input)

print("User Input:")
print(user_input)

print("\nDetected Symptoms:")
print(detected)
```

```
User Input:
I have fever, headache, chills and joint pain

Detected Symptoms:
['high_fever', 'mild_fever', 'headache', 'joint_pain', 'chills']
```

In [26]:
```
# ========================================
# STEP 12E: CREATE ML INPUT VECTOR
# ========================================

def create_symptom_vector(detected_symptoms):

    # Create all-zero vector
    symptom_vector = np.zeros(len(symptom_columns))

    # Set detected symptoms to 1
    for symptom in detected_symptoms:

        if symptom in symptom_columns:

            index = symptom_columns.index(symptom)
            symptom_vector[index] = 1

    # Convert to DataFrame
    symptom_df = pd.DataFrame(
        [symptom_vector],
        columns=symptom_columns
    )

    return symptom_df
```

In [27]:
```
# ========================================
# STEP 12F: TEST DISEASE PREDICTION
# ========================================

user_input = "I have fever, headache, chills and joint pain"

# Extract symptoms
detected_symptoms = extract_symptoms(user_input)

# Create model input
input_vector = create_symptom_vector(detected_symptoms)

# Predict disease
prediction = best_model.predict(input_vector)

# Convert prediction back to disease name
predicted_disease = label_encoder.inverse_transform(prediction)[0]

print("User Input:")
print(user_input)

print("\nDetected Symptoms:")
print(detected_symptoms)

print("\nPredicted Disease:")
print(predicted_disease)
```

```
User Input:
I have fever, headache, chills and joint pain

Detected Symptoms:
['high_fever', 'mild_fever', 'headache', 'joint_pain', 'chills']

Predicted Disease:
Malaria
```

In [28]:
```
# Check all fever-related symptoms in the dataset

fever_symptoms = [
    symptom for symptom in symptom_columns
    if "fever" in symptom.lower()
]

print("Fever-related symptoms:")
print(fever_symptoms)
```

```
Fever-related symptoms:
['high_fever', 'mild_fever']
```

In [29]:
```
user_input = "I have fever, headache, chills and joint pain"

detected_symptoms = extract_symptoms(user_input)

print("User Input:")
print(user_input)

print("\nDetected Symptoms:")
print(detected_symptoms)
```

```
User Input:
I have fever, headache, chills and joint pain

Detected Symptoms:
['high_fever', 'mild_fever', 'headache', 'joint_pain', 'chills']
```

In [30]:
```
input_vector = create_symptom_vector(detected_symptoms)

prediction = best_model.predict(input_vector)

predicted_disease = label_encoder.inverse_transform(prediction)[0]

print("User Input:")
print(user_input)

print("\nDetected Symptoms:")
print(detected_symptoms)

print("\nPredicted Disease:")
print(predicted_disease)
```

```
User Input:
I have fever, headache, chills and joint pain

Detected Symptoms:
['high_fever', 'mild_fever', 'headache', 'joint_pain', 'chills']

Predicted Disease:
Malaria
```

In [31]:
```
import pandas as pd

# Load recommendation datasets

medications = pd.read_csv(r"C:\Users\Manaswini\Downloads\adst\medications.csv")
description = pd.read_csv(r"C:\Users\Manaswini\Downloads\adst\description.csv")
precautions = pd.read_csv(r"C:\Users\Manaswini\Downloads\adst\precautions_df.csv")
diets = pd.read_csv(r"C:\Users\Manaswini\Downloads\adst\diets.csv")
workout = pd.read_csv(r"C:\Users\Manaswini\Downloads\adst\workout_df.csv")

print("Datasets loaded successfully!")
```

```
Datasets loaded successfully!
```

In [32]:
```
print("MEDICATIONS")
display(medications.head())

print("\nDESCRIPTION")
display(description.head())

print("\nPRECAUTIONS")
display(precautions.head())

print("\nDIETS")
display(diets.head())

print("\nWORKOUT")
display(workout.head())
```

```
MEDICATIONS
```

	Disease	Medication
0	Fungal infection	['Antifungal Cream', 'Fluconazole', 'Terbinafi...
1	Allergy	['Antihistamines', 'Decongestants', 'Epinephri...
2	GERD	['Proton Pump Inhibitors (PPIs)', 'H2 Blockers...
3	Chronic cholestasis	['Ursodeoxycholic acid', 'Cholestyramine', 'Me...
4	Drug Reaction	['Antihistamines', 'Epinephrine', 'Corticoster...

```
DESCRIPTION
```

	Disease	Description
0	Fungal infection	Fungal infection is a common skin condition ca...
1	Allergy	Allergy is an immune system reaction to a subs...
2	GERD	GERD (Gastroesophageal Reflux Disease) is a di...
3	Chronic cholestasis	Chronic cholestasis is a condition where bile ...
4	Drug Reaction	Drug Reaction occurs when the body reacts adve...

```
PRECAUTIONS
```

	Unnamed: 0	Disease	Precaution_1	Precaution_2	Precaution_3	Precaution_4
0	0	Drug Reaction	stop irritation	consult nearest hospital	stop taking drug	follow up
1	1	Malaria	Consult nearest hospital	avoid oily food	avoid non veg food	keep mosquitos out
2	2	Allergy	apply calamine	cover area with bandage	NaN	use ice to compress itching
3	3	Hypothyroidism	reduce stress	exercise	eat healthy	get proper sleep
4	4	Psoriasis	wash hands with warm soapy water	stop bleeding using pressure	consult doctor	salt baths

```
DIETS
```

	Disease	Diet
0	Fungal infection	['Antifungal Diet', 'Probiotics', 'Garlic', 'C...
1	Allergy	['Elimination Diet', 'Omega-3-rich foods', 'Vi...
2	GERD	['Low-Acid Diet', 'Fiber-rich foods', 'Ginger'...
3	Chronic cholestasis	['Low-Fat Diet', 'High-Fiber Diet', 'Lean prot...
4	Drug Reaction	['Antihistamine Diet', 'Omega-3-rich foods', '...

WORKOUT

	Unnamed: 0.1	Unnamed: 0	disease	workout
0	0	0	Fungal infection	Avoid sugary foods
1	1	1	Fungal infection	Consume probiotics
2	2	2	Fungal infection	Increase intake of garlic
3	3	3	Fungal infection	Include yogurt in diet
4	4	4	Fungal infection	Limit processed foods

In [34]:
```
print("Medications columns:", medications.columns.tolist())
print("Description columns:", description.columns.tolist())
print("Precautions columns:", precautions.columns.tolist())
print("Diets columns:", diets.columns.tolist())
print("Workout columns:", workout.columns.tolist())
```

```
Medications columns: ['Disease', 'Medication']
Description columns: ['Disease', 'Description']
Precautions columns: ['Unnamed: 0', 'Disease', 'Precaution_1', 'Precaution_2', 'Precaution_3', 'Precaution_4']
Diets columns: ['Disease', 'Diet']
Workout columns: ['Unnamed: 0.1', 'Unnamed: 0', 'disease', 'workout']
```

In [35]:
```
predicted_disease = "Malaria"

print("==================================")
print("PREDICTED DISEASE:", predicted_disease)
print("==================================")


# 1. DESCRIPTION
description_result = description[
    description["Disease"].astype(str).str.lower() == predicted_disease.lower()
]

print("\n--- DESCRIPTION ---")
print(description_result["Description"].to_string(index=False))


# 2. MEDICATION
medication_result = medications[
    medications["Disease"].astype(str).str.lower() == predicted_disease.lower()
]

print("\n--- MEDICATION ---")
print(medication_result["Medication"].to_string(index=False))


# 3. PRECAUTIONS
precaution_result = precautions[
    precautions["Disease"].astype(str).str.lower() == predicted_disease.lower()
]

print("\n--- PRECAUTIONS ---")

if not precaution_result.empty:
    for column in ["Precaution_1", "Precaution_2", "Precaution_3", "Precaution_4"]:
        value = precaution_result[column].iloc[0]

        if pd.notna(value):
            print("•", value)


# 4. DIET
diet_result = diets[
    diets["Disease"].astype(str).str.lower() == predicted_disease.lower()
]

print("\n--- DIET ---")
print(diet_result["Diet"].to_string(index=False))


# 5. WORKOUT / LIFESTYLE
workout_result = workout[
    workout["disease"].astype(str).str.lower() == predicted_disease.lower()
]

print("\n--- WORKOUT / LIFESTYLE ---")
print(workout_result["workout"].to_string(index=False))
```

```
==================================
PREDICTED DISEASE: Malaria
==================================

--- DESCRIPTION ---
Malaria is a mosquito-borne infectious disease ...

--- MEDICATION ---
['Antimalarial drugs', 'Antipyretics', 'Antieme...

--- PRECAUTIONS ---
• Consult nearest hospital
• avoid oily food
• avoid non veg food
• keep mosquitos out

--- DIET ---
['Malaria Diet', 'Hydration', 'High-Calorie Die...

--- WORKOUT / LIFESTYLE ---
                    Stay hydrated
      Consume nutrient-rich foods
       Include protein-rich foods
Consume foods rich in antioxidants
     Limit fatty and greasy foods
       Avoid alcohol and caffeine
     Include vitamin C-rich foods
 Consult a healthcare professional
         Rest and conserve energy
     Gradually resume normal diet
```

In [37]:
```
def predict_disease_and_recommendations(user_input):
```

```
    # Step 1: Extract symptoms from natural language
    detected_symptoms = extract_symptoms(user_input)

    # Step 2: Convert symptoms into model input
    input_vector = create_symptom_vector(detected_symptoms)

    # Step 3: Predict disease
    prediction = best_model.predict(input_vector)

    predicted_disease = label_encoder.inverse_transform(prediction)[0]

    # Step 4: Get description
    description_result = description[
        description["Disease"].astype(str).str.lower()
        == predicted_disease.lower()
    ]

    disease_description = ""

    if not description_result.empty:
        disease_description = description_result["Description"].iloc[0]

    # Step 5: Get medication information
    medication_result = medications[
        medications["Disease"].astype(str).str.lower()
        == predicted_disease.lower()
    ]

    medicine_info = []

    if not medication_result.empty:
        medicine_info = medication_result["Medication"].iloc[0]

    # Step 6: Get precautions
    precaution_result = precautions[
        precautions["Disease"].astype(str).str.lower()
        == predicted_disease.lower()
    ]

    precaution_list = []

    if not precaution_result.empty:
        for column in [
            "Precaution_1",
            "Precaution_2",
            "Precaution_3",
            "Precaution_4"
        ]:
            value = precaution_result[column].iloc[0]

            if pd.notna(value):
                precaution_list.append(value)

    # Step 7: Get diet
    diet_result = diets[
        diets["Disease"].astype(str).str.lower()
        == predicted_disease.lower()
    ]

    diet_info = []

    if not diet_result.empty:
        diet_info = diet_result["Diet"].iloc[0]

    # Step 8: Get lifestyle/workout
    workout_result = workout[
        workout["disease"].astype(str).str.lower()
        == predicted_disease.lower()
    ]

    workout_info = []

    if not workout_result.empty:
        workout_info = workout_result["workout"].tolist()

    # Return everything
    return {
        "input": user_input,
        "symptoms": detected_symptoms,
        "disease": predicted_disease,
        "description": disease_description,
        "medication": medicine_info,
        "precautions": precaution_list,
        "diet": diet_info,
        "workout": workout_info
    }
```

In [38]:

```
user_input = "I have fever, headache, chills and joint pain"

result = predict_disease_and_recommendations(user_input)

print("USER INPUT:")
print(result["input"])

print("\nDETECTED SYMPTOMS:")
print(result["symptoms"])

print("\nPREDICTED DISEASE:")
print(result["disease"])

print("\nDESCRIPTION:")
print(result["description"])

print("\nMEDICATION:")
print(result["medication"])

print("\nPRECAUTIONS:")
for item in result["precautions"]:
    print("•", item)

print("\nDIET:")
print(result["diet"])

print("\nLIFESTYLE / WORKOUT:")
for item in result["workout"]:
    print("•", item)
```

```
USER INPUT:
I have fever, headache, chills and joint pain

DETECTED SYMPTOMS:
['high_fever', 'mild_fever', 'headache', 'joint_pain', 'chills']

PREDICTED DISEASE:
Malaria

DESCRIPTION:
Malaria is a mosquito-borne infectious disease affecting humans and other animals.

MEDICATION:
['Antimalarial drugs', 'Antipyretics', 'Antiemetic drugs', 'IV fluids', 'Blood transfusions']

PRECAUTIONS:
• Consult nearest hospital
• avoid oily food
• avoid non veg food
• keep mosquitos out

DIET:
['Malaria Diet', 'Hydration', 'High-Calorie Diet', 'Soft and bland foods', 'Oral rehydration solutions']

LIFESTYLE / WORKOUT:
• Stay hydrated
• Consume nutrient-rich foods
• Include protein-rich foods
• Consume foods rich in antioxidants
• Limit fatty and greasy foods
• Avoid alcohol and caffeine
• Include vitamin C-rich foods
• Consult a healthcare professional
• Rest and conserve energy
• Gradually resume normal diet
```

In [39]:
```
import joblib

joblib.dump(best_model, "model.pkl")
joblib.dump(label_encoder, "label_encoder.pkl")
joblib.dump(symptom_columns, "symptom_columns.pkl")
joblib.dump(symptom_mapping, "symptom_mapping.pkl")

print("Model and NLP files saved successfully!")
```

```
Model and NLP files saved successfully!
```

In [40]:
```
import os
print(os.getcwd())
```

```
C:\Users\Manaswini\Documents\ADST
```

In [42]:
```

```

```
C:\Users\Manaswini\Documents\ADST
['.ipynb_checkpoints', 'label_encoder.pkl', 'Medicine_Prediction_Project.ipynb.ipynb', 'model.pkl', 'symptom_columns.pkl', 'symptom_mapping.pkl', 'Untitled.ipynb', 'untitled.txt', 'Untitled1.ipynb', 'Untitled2.ipynb', 'Untitled3.ipynb']
```

In [ ]:
```

```